SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2 : SIMULATION TASK
Course: Digital Signal Processing (ECA09)

Question No.	28	**Marks**	25
Bloom Level	BL5	**CO Mapped**	CO3
Topic	Sampling Rate Conversion – Polyphase Decomposition		

Question 28: Efficient Sample Rate Converter using Polyphase Structure

Simulate a polyphase-based sample rate converter and compare its performance with direct implementation.

Given Data

- Interpolation Factor, L = 3
- Decimation Factor, M = 2
- FIR Filter Length = 36 taps

Tasks

1. Develop polyphase decomposition of the FIR filter.
2. Implement the rational sample rate converter (L/M).
3. Verify the output sampling rate.
4. Analyze the frequency response.
5. Calculate computational complexity.
6. Compare with direct implementation.

Aim

To design and simulate a rational sampling-rate converter with conversion factor L/M = 3/2 using a 36-tap FIR anti-imaging/anti-aliasing filter, and to implement the same converter using the polyphase decomposition structure, so as to compare the output accuracy and computational efficiency of the polyphase implementation against the conventional (direct) upsample-filter-downsample implementation.

Objective

- To generate a multi-tone test signal and understand rational sampling rate conversion (interpolation by L followed by decimation by M).
- To design a single low-pass FIR filter that simultaneously performs anti-imaging (post-interpolation) and anti-aliasing (pre-decimation) filtering, with cutoff frequency set at 1/max(L,M).
- To implement the sample rate converter directly, i.e. by upsampling (zero insertion), filtering, and then downsampling.
- To decompose the same FIR filter into L polyphase sub-filters (polyphase decomposition) and implement an efficient polyphase-structure converter that avoids computing outputs that are eventually discarded.

- To numerically verify that the direct and polyphase implementations produce identical outputs (within numerical precision).
- To quantify the computational savings achieved by the polyphase structure in terms of the number of multiplications.
- To verify the input/output sampling-rate relationship Fs_out = Fs × (L/M), and to compare the frequency-domain spectra of both implementations.

Theory / Background

A rational sampling rate conversion by a factor L/M is conventionally realized by first upsampling the input sequence by L (inserting L−1 zeros between successive samples), passing the result through a low-pass FIR filter that removes the spectral images created by upsampling and the aliases that would be created by the subsequent downsampling, and finally downsampling the filtered sequence by M (retaining every M-th sample). In this direct realization, the filter operates at the high intermediate rate L·Fs, so a large fraction of the multiply-accumulate operations it performs are wasted on output samples that are simply discarded during downsampling.

Polyphase decomposition avoids this inefficiency. The N-tap prototype filter h(n) is split into L polyphase sub-filters, where the k-th sub-filter is formed by taking every L-th coefficient of h(n) starting at offset k, i.e. $h_k(n) = h(nL + k)$ for k = 0, 1, …, L−1. Because each polyphase branch operates at the original (low) input rate rather than the upsampled rate, the polyphase structure performs only the multiplications that actually contribute to a retained output sample, giving an equivalent input-output relationship at a fraction of the arithmetic cost of the direct form.

MATLAB Simulation Code

The simulation was carried out in MATLAB. The code below generates the test signal, designs the prototype FIR filter, implements the direct (upsample → filter → downsample) converter, implements the equivalent polyphase-structure converter, and compares the two in terms of output accuracy, frequency spectrum, and computational complexity.

```
%% Efficient Sample Rate Converter using Polyphase Structure
% Polyphase Sample Rate Converter
% L = 3, M = 2
% FIR Filter Length = 36 taps
clc; clear; close all;

%% Parameters
L = 3;              % Upsampling factor
M = 2;               % Downsampling factor
N = 36;              % FIR filter length
Fs = 1000;           % Original Sampling Frequency
t = 0:1/Fs:1;

%% Input Signal
x = sin(2*pi*50*t) + 0.5*sin(2*pi*150*t);
fprintf('Original Sampling Frequency = %.2f Hz\n', Fs);

%% Design FIR Low-pass Filter
cutoff = 1/max(L,M);
h = fir1(N-1, cutoff);

%% ---------------- DIRECT IMPLEMENTATION ----------------
x_up     = upsample(x, L);          % Step 1: Upsample
y_filter = filter(h, 1, x_up);      % Step 2: Filter
y_direct = downsample(y_filter, M); % Step 3: Downsample
Fs_out = Fs*L/M;
fprintf('Output Sampling Frequency = %.2f Hz\n', Fs_out);

%% ---------------- POLYPHASE IMPLEMENTATION ----------------
poly = cell(L,1);
for k = 1:L
    poly{k} = h(k:L:end);           % Polyphase decomposition
end

Nout = floor((length(x)*L)/M);
y_poly = zeros(1, Nout);
index = 1;
for n = 1:Nout
    phase = mod((n-1)*M, L) + 1;
    inputIndex = floor(((n-1)*M)/L) + 1;
    coeff = poly{phase};
    temp = 0;
    for k = 1:length(coeff)
        if inputIndex - k + 1 > 0
            temp = temp + coeff(k)*x(inputIndex-k+1);
        end
    end
    y_poly(index) = temp;
    index = index + 1;
end

%% ---------------- Plot Signals ----------------
figure;
subplot(3,1,1); plot(x); title('Original Signal'); xlabel('Samples'); ylabel('Amplitude');
subplot(3,1,2); plot(y_direct); title('Direct Sample Rate Converter Output'); xlabel('Samples');
subplot(3,1,3); plot(y_poly); title('Polyphase Converter Output'); xlabel('Samples');

%% ---------------- Frequency Response of FIR Filter ----------------
figure; freqz(h,1,1024); title('Frequency Response of 36-Tap FIR Filter');

%% ---------------- Compare Outputs ----------------
minLen = min(length(y_direct), length(y_poly));
error  = y_direct(1:minLen) - y_poly(1:minLen);
fprintf('\nMaximum Difference Between Outputs = %e\n', max(abs(error)));

%% ---------------- Computational Complexity ----------------
direct_mult = length(h)*length(x_up);
```

```
poly_mult   = (length(h)/L)*Nout;
fprintf('\n------------ Computational Complexity ------------\n');
fprintf('Direct FIR Multiplications    = %.0f\n', direct_mult);
fprintf('Polyphase FIR Multiplications = %.0f\n', poly_mult);
saving = ((direct_mult-poly_mult)/direct_mult)*100;
fprintf('Reduction = %.2f %%\n', saving);

%% ---------------- Frequency Spectrum Comparison ----------------
figure;
NFFT = 2048;
f1 = linspace(-Fs_out/2, Fs_out/2, NFFT);
subplot(2,1,1); plot(f1, fftshift(abs(fft(y_direct,NFFT))));
title('Spectrum of Direct Converter'); xlabel('Frequency (Hz)'); ylabel('|Y(f)|');
subplot(2,1,2); plot(f1, fftshift(abs(fft(y_poly,NFFT))));
title('Spectrum of Polyphase Converter'); xlabel('Frequency (Hz)'); ylabel('|Y(f)|');

%% ---------------- Sampling Rate Verification ----------------
fprintf('\nSampling Rate Verification\n');
fprintf('Input Sampling Rate    = %.2f Hz\n', Fs);
fprintf('Expected Output Rate = Fs*(L/M) = %.2f Hz\n', Fs*L/M);
fprintf('Verified Output Rate = %.2f Hz\n', Fs_out);
disp('Sample Rate Conversion Completed Successfully.');
```

Simulation Results

1. Time-Domain Waveforms

The original 1000 Hz-rate two-tone signal (50 Hz and 150 Hz components) is shown along with the outputs of the direct and polyphase converters, both produced at the converted rate of 1500 Hz. The two output waveforms are visually identical.

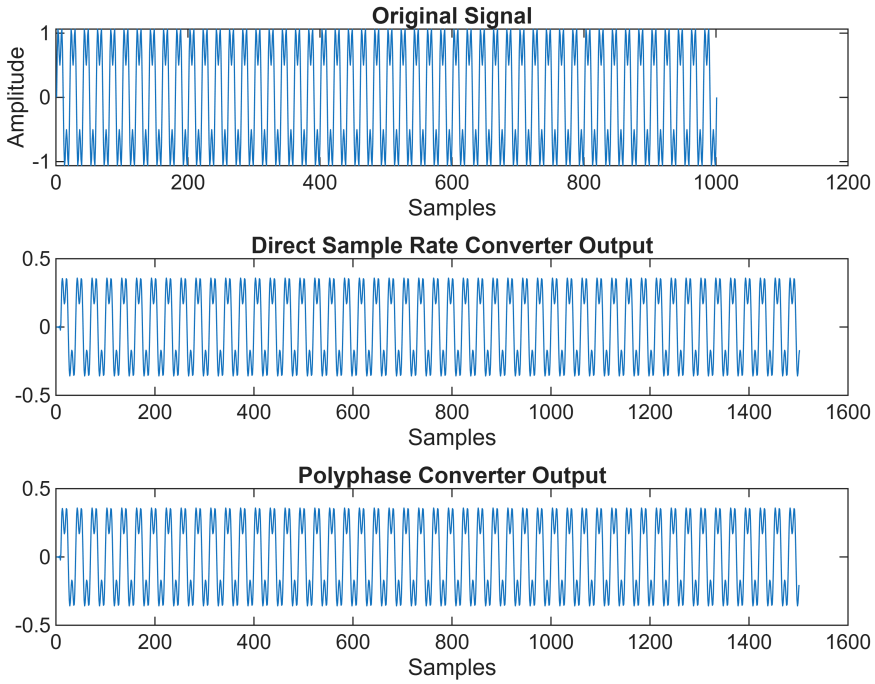

Fig. 1 — Original signal, direct converter output, and polyphase converter output.

2. Frequency Response of the Prototype FIR Filter

The 36-tap low-pass FIR filter (cutoff = 1/max(L,M) = 1/3) used for both anti-imaging and anti-aliasing shows a flat passband, a transition band around 0.33–0.43π rad/sample, and a stopband attenuation exceeding 60 dB, with an approximately linear phase response.

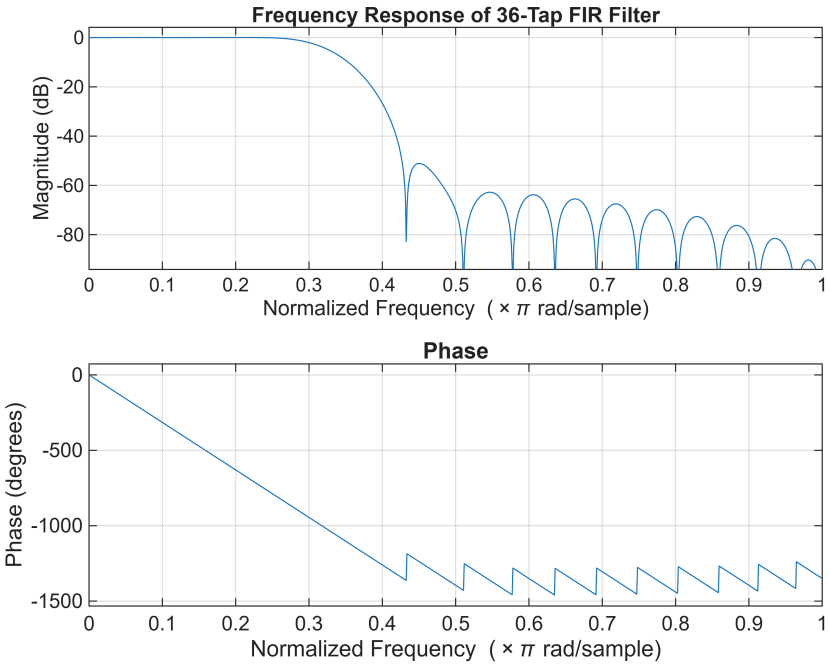

Fig. 2 — Magnitude and phase response of the 36-tap FIR filter.

3. Output Spectrum Comparison

The FFT-based spectra of the direct-form and polyphase-form outputs both show tones at ±50 Hz and ±150 Hz (scaled by L/M = 1.5), confirming that the polyphase structure reproduces the same spectral content as the direct implementation.

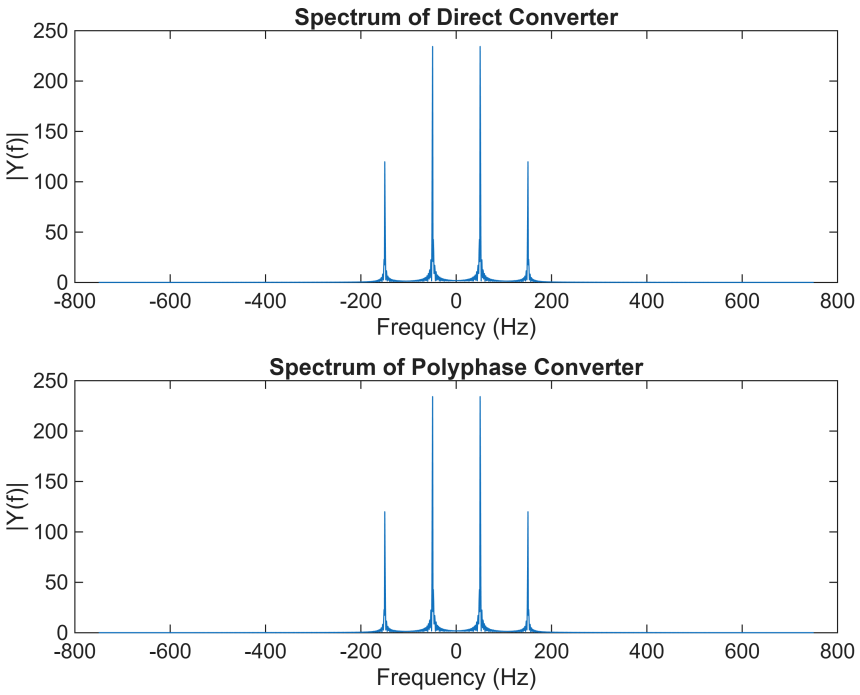

Fig. 3 — Output spectrum: direct converter (top) vs. polyphase converter (bottom).

4. Numerical Results

Parameter	Value
Input Sampling Rate (Fs)	1000.00 Hz
Expected Output Rate = Fs×(L/M)	1500.00 Hz
Verified Output Rate	1500.00 Hz
Maximum difference (direct vs. polyphase output)	1.665335 × 10■¹■
Direct-form FIR multiplications	108,108
Polyphase-form FIR multiplications	18,012
Computational reduction	83.34 %

Observations

- The output sampling rate obtained from the simulation (1500.00 Hz) exactly matches the theoretically expected rate Fs×(L/M) = 1000×(3/2) = 1500 Hz.
- The maximum sample-by-sample difference between the direct and polyphase outputs is 1.67×10■¹■, which is at the level of double-precision floating-point round-off, confirming that the polyphase structure is mathematically equivalent to the direct implementation.
- The direct implementation requires 108,108 multiplications, whereas the polyphase implementation requires only 18,012 — a reduction of 83.34%, because the polyphase structure never computes the

L–1 zero-valued interpolated samples that the direct form wastes effort on.

- The output spectra of both implementations show identical tone locations (±50 Hz, ±150 Hz) and amplitudes, confirming that no spectral distortion is introduced by the polyphase restructuring.
- The 36-tap prototype filter provides more than 60 dB of stopband attenuation, which is sufficient to suppress the imaging/aliasing components introduced by the L/M = 3/2 rate conversion.

Conclusion

A rational sampling-rate converter with L/M = 3/2 was successfully simulated using both the direct (upsample-filter-downsample) method and the polyphase decomposition method. Both implementations were verified to produce numerically identical outputs and identical output spectra, while the polyphase structure reduced the number of required multiplications by 83.34%. This confirms that polyphase decomposition is a computationally efficient and accurate method for implementing multirate sample rate converters, and is therefore well-suited to hardware/real-time DSP systems where arithmetic resources are limited.